

LAB3 실험 전 보고서

LAB3 - 조합논리 7개

실험 주제 : LAB3 조합논리 7개 - 입력 조합 및 출력 관계 분석

실험 대상 : LED_PWM, RGB_PWM, PIEZO, STEPPER, MMSS_CLOCK,
CHARACTER_LCD, UART_ECHO

작성일 : 2026년 9월 27일

검증 환경 : VS Code Vapor View

성명 : 권진혁

학과 : 전자전기컴퓨터공학부

학번 : 2023440007

1. LED_PWM

· 동작 원리와 핵심 파라미터 계산

본 실험은 버튼을 누를 때마다 PWM(Pulse Width Modulation)의 듀티비(Duty Cycle)를 변경하여 LED 밝기를 조절하는 회로를 구현하는 것이다. PWM은 신호의 주기는 일정하게 유지하면서 HIGH 구간의 비율만 조절하는 방식이다. HIGH 구간이 길수록 LED에 공급되는 평균 전력이 증가하므로 LED가 더 밝게 보인다.

· 버튼 처리 과정

버튼 입력은 기계식 접점의 바운스 현상이 존재하므로 다음 과정을 거친다.

- 2단 동기화(button_meta, button_sync)
- 디바운싱(STABLE_CYCLES)
- One-Pulse 생성
- level 값 증가

버튼이 한 번 눌리면 press 신호가 정확히 1클록 동안만 발생한다. 이 신호를 이용해 PWM 레벨을 변경한다.

· PWM 주기 계산

입력 클록: 50MHz, PWM 주파수: 1kHz이므로 PWM 한 주기는 50,000개의 클록으로 구성된다.

· 듀티비 계산

설계는 LEVELS=10으로 설정되어 있다.

level	Duty	HIGH 클록 수
0	0%	0
1	10%	5,000
3	30%	15,000
5	50%	25,000
10	100%	50,000

따라서 버튼을 누를 때마다 LED 밝기가 약

10%씩 증가하며, 100% 다음에는 다시 0%로 순환한다.

· RTL

```
src > button_onepulse.v > {} button_onepulse > always > {} if > {} else > {} if > {} else if > {} else
1  `timescale 1ns/1ps
2
3  module button_onepulse #(
4      parameter integer STABLE_CYCLES = 1_000_000
5  )(
6      input wire clk,
7      input wire rst_p,
8      input wire button,
9      output reg pulse
10 );
11
12 (* ASYNC_REG = "TRUE" *) reg button_meta;
13 (* ASYNC_REG = "TRUE" *) reg button_sync;
14
15 localparam integer COUNT_WIDTH =
16     (STABLE_CYCLES < 2) ? 1 : $clog2(STABLE_CYCLES);
17
18 reg [COUNT_WIDTH-1:0] stable_count;
19 reg accepted;
20
21 always @(posedge clk or posedge rst_p) begin
22     if (rst_p) begin
23         button_meta <= 1'b0;
24         button_sync <= 1'b0;
25     end else begin
26         button_meta <= button;
27         button_sync <= button_meta;
28     end
29 end
30
31 always @(posedge clk or posedge rst_p) begin
32     if (rst_p) begin
33         stable_count <= {COUNT_WIDTH{1'b0}};
34         accepted <= 1'b0;
35         pulse <= 1'b0;
36     end else begin
37         pulse <= 1'b0;
38
39         if (button_sync == accepted) begin
40             stable_count <= {COUNT_WIDTH{1'b0}};
41         end else if (stable_count == STABLE_CYCLES - 1) begin
42             stable_count <= {COUNT_WIDTH{1'b0}};
43             accepted <= button_sync;
44             pulse <= button_sync;
45         end else begin
46             stable_count <= stable_count + 1'b1;
47         end
48     end
49 end
50
51 endmodule
```

- 버튼 입력 동기화
- 디바운싱 처리
- 버튼 1회 입력당 1클록 펄스 생성

```

src > lab3_led_pwm.v 2,U  pwmm_channel.v 7,U  tb_led_pwm.sv 1,U  lab3_led_pwm.xdc U
1 `timescale 1ns/1ps
2
3 module pwmm_channel #(
4     parameter integer PERIOD_CYCLES = 50_000,
5     parameter integer LEVELS = 10
6 )
7     input wire clk,
8     input wire rst_p,
9     input wire [3:0] level,
10    output reg pwm
11;
12
13    localparam integer COUNT_WIDTH =
14        (PERIOD_CYCLES < 2) ? 1 : $clog2(PERIOD_CYCLES);
15
16    reg [COUNT_WIDTH-1:0] count;
17    integer threshold;
18
19    always @* begin
20        if (level >= LEVELS)
21            threshold = PERIOD_CYCLES;
22        else
23            threshold = (PERIOD_CYCLES * level) / LEVELS;
24        end
25
26    always @(posedge clk or posedge rst_p) begin
27        if (rst_p) begin
28            count <= {COUNT_WIDTH{1'b0}};
29            pwm <= 1'b0;
30        end else begin
31            pwm <= (count < threshold);
32            if (count == PERIOD_CYCLES - 1)
33                count <= {COUNT_WIDTH{1'b0}};
34            else
35                count <= count + 1'b1;
36        end
37    end
38 endmodule

```

- PWM 파형 생성
- level 값에 따라 threshold 계산
- count < threshold 동안 pwm=1 출력

```

src > lab3_led_pwm.v 2,U  pwmm_channel.v 7,U  tb_led_pwm.sv 1,U  lab3_led_pwm.xdc U  {} sim
1 `timescale 1ns/1ps
2
3 module lab3_led_pwm #(
4     parameter integer CLK_HZ = 50_000_000,
5     parameter integer PWM_HZ = 1_000,
6     parameter integer LEVELS = 10,
7     parameter integer DEBOUNCE_CYCLES = 1_000_000
8 )
9     input wire clk_50mhz,
10    input wire rst_p,
11    input wire button,
12    output wire [7:0] led
13;
14
15    wire press;
16    wire pwm;
17    reg [3:0] level;
18
19    button_onepulse #(
20        .STABLE_CYCLES(DEBOUNCE_CYCLES)
21    ) u_button (
22        .clk(clk_50mhz),
23        .rst_p(rst_p),
24        .button(button),
25        .pulse(press)
26    );
27
28    always @(posedge clk_50mhz or posedge rst_p) begin
29        if (rst_p)
30            level <= 4'd0;
31        else if (press)
32            level <= (level == LEVELS) ? 4'd0 : level + 1'b1;
33        end
34
35    pwmm_channel #(
36        .PERIOD_CYCLES(CLK_HZ / PWM_HZ),
37        .LEVELS(LEVELS)
38    )

```

```

39    ) u_pwm (
40        .clk(clk_50mhz),
41        .rst_p(rst_p),
42        .level(level),
43        .pwm(pwm)
44    );
45
46    assign led = {8{pwm}};
47
48 endmodule

```

- 전체 최상위 모듈
- 버튼 입력 처리
- level 값 증가 및 순환
- PWM 출력 생성
- 8개 LED에 동일 PWM 출력
- 즉 모든 LED가 같은 밝기로 동작한다.

· RTL TOP MODULE

‘module lab3_led_pwm’

· TB

```

sim > tb_led_pwm.sv > {} tb_led_pwm > initial
1 `timescale 1ns/1ps
2
3 module tb_led_pwm;
4
5     reg clk_50mhz=0, rst_p=1, button=0;
6     wire [7:0] led;
7     integer checks=0, high_count, i;
8
9     always #10 clk_50mhz=~clk_50mhz;
10
11    lab3_led_pwm #(
12        .CLK_HZ(1000),
13        .PWM_HZ(100),
14        .LEVELS(10),
15        .DEBOUNCE_CYCLES(2)
16    ) dut (
17        .clk_50mhz(clk_50mhz),
18        .rst_p(rst_p),
19        .button(button),
20        .led(led)
21    );
22
23    task press_button;
24    begin
25        button=1;
26        repeat(6) @(posedge clk_50mhz);
27        button=0;
28        repeat(6) @(posedge clk_50mhz);
29    end
30    endtask
31
32    task expect_level(input integer expected);
33    begin
34        wait(dut.u_pwm.count==0);
35        high_count=0;
36
37        for(i=0;i<10;i=i+1) begin
38            @(posedge clk_50mhz);

```

- 클록 생성
- 버튼 입력 생성
- expected level 검사
- PASS/FAIL 자동 판정
- wave.vcd 생성

- Simulation Top Module

```
'module tb_led_pwm'
```

- XDC

```

1  set_property PACKAGE_PIN B6 [get_ports clk_50mhz]
2
3  set_property PACKAGE_PIN K4 [get_ports rst_p]
4
5  set_property PACKAGE_PIN N8 [get_ports button]
6
7  set_property PACKAGE_PIN N5 [get_ports {led[0]}]
8  set_property PACKAGE_PIN M1 [get_ports {led[1]}]
9  set_property PACKAGE_PIN M3 [get_ports {led[2]}]
10 set_property PACKAGE_PIN M7 [get_ports {led[3]}]
11 set_property PACKAGE_PIN N7 [get_ports {led[4]}]
12 set_property PACKAGE_PIN M2 [get_ports {led[5]}]
13 set_property PACKAGE_PIN M4 [get_ports {led[6]}]
14 set_property PACKAGE_PIN L4 [get_ports {led[7]}]
15
16 set_property IOSTANDARD LVCMOS33 [get_ports *]
17
18 create_clock -name clk_50mhz -period 20.000 [get_ports clk_50mhz]
19
20 set_false_path -from [get_ports {rst_p button}]

```

- FPGA 핀 배치
- 50MHz 클록 정의
- I/O 전압 규격 지정
- 타이밍 제약 설정

· TB 자극과 기대 결과

TB에서는 총 4개의 기능을 검사한다.

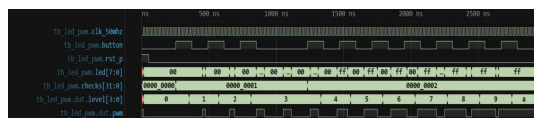
검사 순서	버튼 입력	예상 level	PWM HIGH 개수
1	초기 상태	0	0
2	3회 입력	3	3
3	추가 7회 입력	10	10
4	추가 1회 입력	0	0

한 PWM 주기에서 HIGH 클록 수를 세어
기대값과 비교한다.

· PASS Log

[illegible]

· 파형 분석



- Level
 $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$
 $\rightarrow 8 \rightarrow 9 \rightarrow A$
 순서로 증가한다.

- PWM

level 증가에 따라 HIGH 구간 폭이 점차 길어진다.

level=0 → 거의 HIGH 없음

level=5 → 약 50%

level=A(10) → 항상 HIGH

따라서 PWM 듀티비가 정상적으로 증가하는 것을 확인할 수 있다.

- Checks 신호

0 → 1 → 2

검증 단계가 진행되고 있음을 보여준다.

· 경계 조건

조건	기대 결과
level=0	PWM 항상 LOW
level=10	PWM 항상 HIGH
level=10에서 버튼 입력	level=0 복귀
버튼 바운스 발생	one-pulse로 1회만 인식
reset 입력	level=0, pwm=0

· 수정 실험 및 복구

‘LEVELS = 10’에서 10을 5로 바꾼다.



수정 시 듀티 단계가 10단계에서 5단계로 줄어들므로 파형 HIGH 폭 변화가 더 크게 나타난다.

상태	결과
정상	LEVELS=10, PASS
수정	LEVELS=5, PWM 단계 수 감소, 기대값 불일치 가능
복구	LEVELS=10 복원 후 PASS

· 보드에서 확인해야 할 항목과 예상 결과

확인 항목 1

- 버튼 1회 입력

- 예상 결과

level이 1 증가

LED 밝기 약 10% 증가

확인 항목 2

- 중간 단계 밝기

- 예상 결과

20%, 30%, 40%, 50% 순으로 밝기

증가

육안으로 밝기 차이 확인 가능

확인 항목 3

- 100% 상태

- 예상 결과

모든 LED가 항상 켜짐

깜빡임 없음

확인 항목 4

- 100% 이후 버튼 입력

- 예상 결과

level=0 복귀

LED 모두 꺼짐

확인 항목 5

- Reset 동작

- 예상 결과

level=0

pwm=0

모든 LED OFF

· 최종 예상 결과

버튼을 누를 때마다 LED 밝기가 순환하며, 8개의 LED가 모두 동일한 밝기로 동작한다.

시뮬레이션에서는 'LAB3_LED_PWM_PASS checks=4'가 출력되고, 실제 FPGA 보드에 서도 같은 동작이 관찰될 것으로 예상된다.

2. RGB_PWM

· 동작 원리와 핵심 파라미터 계산

본 회로는 RGB LED의 밝기를 PWM(Pulse Width Modulation) 방식으로 제어하는 시스템이다. 사용자가 R, G, B 버튼을 누르면 해당 색상의 duty cycle이 증가하며, PWM 출력의 HIGH 비율에 따라 LED 밝기가 달라진다. 세 색의 밝기를 독립적으로 조절하여 다양한 혼합색을 만들 수 있다.

· PWM 동작 원리

PWM은 일정한 주기(period)를 유지한 상태에서 HIGH 구간의 비율을 조절하는 방식이다. 예를 들어 Duty가 20%이면 한 주기 동안 20% 시간은 HIGH, 나머지 80% 시간은 LOW가 된다.

· 주요 파라미터 계산

- CLK_HZ = 50,000,000 Hz
- PWM_HZ = 1,000 Hz
- LEVELS = 10

PWM 한 주기의 카운트 수는 CLK_HZ / PWM_HZ 이므로 PWM 카운터는 0~49,999까지 동작한다. 각 밝기 단계의 임계값(threshold)은 $(PERIOD_CYCLE * Level) / Levels$ 로 계산된다.

level	duty	threshold
2	20%	10,000
5	50%	25,000
8	80%	40,000
10	100%	50,000

따라서 count 값이 threshold보다 작은 동

안 PWM은 HIGH가 되어 LED가 점등된다.

· 버튼 입력 처리

버튼은 기계적 채터링이 발생하므로 button_onepulse 모듈을 사용하여 디바운싱과 원펄스 생성을 수행한다. 버튼을 한 번 눌러도 level 값이 한 단계만 증가하도록 설계되어 있다.

· TOP Module

'lab3_rgb_pwm'이 모듈이 최상위 설계이며 Vivado 합성 시 Top Module로 선택한다.

· RTL

```

1 timescale 1ns/1ps
2 module button_onepulse #(
3     parameter integer STABLE_CYCLES = 1_000_000
4 )
5     input wire clk,
6     input wire rst_p,
7     input wire button,
8     output reg pulse
9 );
10
11 (* ASYNC_REG = "TRUE" *) reg button_meta;
12 (* ASYNC_REG = "TRUE" *) reg button_sync;
13 localparam integer COUNT_WIDTH = (STABLE_CYCLES < 2) ? 1 : $clog2(STABLE_CYCLES);
14 reg [COUNT_WIDTH-1:0] stable_count;
15 reg accepted;
16
17 always @(posedge clk or posedge rst_p) begin
18     if (rst_p) begin
19         button_meta <- 1'b0;
20         button_sync <- 1'b0;
21     end else begin
22         button_meta <- button;
23         button_sync <- button_meta;
24     end
25 end
26
27 always @(posedge clk or posedge rst_p) begin
28     if (rst_p) begin
29         stable_count <- {COUNT_WIDTH{1'b0}};
30         accepted <- 1'b0;
31         pulse <- 1'b0;
32     end else begin
33         pulse <- 1'b0;
34         if (button_sync == accepted) begin
35             stable_count <- {COUNT_WIDTH{1'b0}};
36         end else if (stable_count == STABLE_CYCLES - 1) begin
37             stable_count <- {COUNT_WIDTH{1'b0}};
38             accepted <- button_sync;
39             pulse <- button_sync;
40         end else begin
41             stable_count <- stable_count + 1'b1;
42         end
43     end
44 end
45 endmodule

```

- 버튼 입력 동기화

- 디바운싱 처리

- 1클럭 폭의 pulse 생성

입력이 여러 번 튀더라도 하나의 버튼 입력으로 인식하게 한다.

```

lab3_rgb_pwm.v 0 X 0 pwm_channel.v 0 X 0 always 0 1 #
src > @ pwm_channel.v (1) pwm_channel > 0 always 0 1 #
1 timescale 1ns/1ps
2 module pwm_channel #(
3     parameter integer PERIOD_CYCLES = 50_000,
4     parameter integer LEVELS = 10
5 )
6     input wire clk,
7     input wire rst_p,
8     input wire [3:0] level,
9     output reg pwm
10 );
11
12 localparam integer COUNT_WIDTH = (PERIOD_CYCLES < 2 ? 1 : $clog2(PERIOD_CYCLES));
13 reg [COUNT_WIDTH-1:0] count;
14 integer threshold;
15
16 always @* begin
17     if (level >= LEVELS)
18         threshold = PERIOD_CYCLES;
19     else
20         threshold = (PERIOD_CYCLES * level) / LEVELS;
21 end
22
23 always @(posedge clk or posedge rst_p) begin
24     if (rst_p) begin
25         count <= {COUNT_WIDTH{1'b0}};
26         pwm <= 1'b0;
27     end else begin
28         pwm <= (count < threshold);
29         if (count == PERIOD_CYCLES - 1)
30             count <= {COUNT_WIDTH{1'b0}};
31         else
32             count <= count + 1'b1;
33     end
34 end
35 endmodule

```

- PWM 카운터 생성
- threshold 계산
- duty cycle에 맞는 PWM 출력 생성
- level 값에 따라 PWM HIGH 폭을 결정한다.

```

lab3_rgb_pwm.v 0 X 0 pwm_channel.v 0 X 0 always 0 1 #
src > @ lab3_rgb_pwm.v (1) lab3_rgb_pwm > 0 always 0 1 #
1 timescale 1ns/1ps
2 module lab3_rgb_pwm #(
3     parameter integer CLK_HZ = 50_000_000,
4     parameter integer PWM_HZ = 1_000,
5     parameter integer LEVELS = 10,
6     parameter integer DEBOUNCE_CYCLES = 1_000_000
7 )
8     input wire clk_50mhz,
9     input wire rst_p,
10    input wire button_r,
11    input wire button_g,
12    input wire button_b,
13    output wire [3:0] led_r,
14    output wire [3:0] led_g,
15    output wire [3:0] led_b
16 );
17    wire press_r, press_g, press_b;
18    wire pwm_r, pwm_g, pwm_b;
19
20    reg [3:0] level_r, level_g, level_b;
21
22    button_onepulse #(.STABLE_CYCLES(DEBOUNCE_CYCLES)) u_button_r (
23        .clk(clk_50mhz), .rst_p(rst_p), .button(button_r), .pulse(press_r));
24    button_onepulse #(.STABLE_CYCLES(DEBOUNCE_CYCLES)) u_button_g (
25        .clk(clk_50mhz), .rst_p(rst_p), .button(button_g), .pulse(press_g));
26    button_onepulse #(.STABLE_CYCLES(DEBOUNCE_CYCLES)) u_button_b (
27        .clk(clk_50mhz), .rst_p(rst_p), .button(button_b), .pulse(press_b));
28
29    always @(posedge clk_50mhz or posedge rst_p) begin
30        if (rst_p) begin
31            level_r <= 0; level_g <= 0; level_b <= 0;
32        end else begin
33            if (press_r) level_r <= (level_r == LEVELS) ? 0 : level_r + 1'b1;
34            if (press_g) level_g <= (level_g == LEVELS) ? 0 : level_g + 1'b1;
35            if (press_b) level_b <= (level_b == LEVELS) ? 0 : level_b + 1'b1;
36        end
37    end
38
39    pwm_channel #(.PERIOD_CYCLES(LEVELS), .LEVELS(LEVELS)) u_pwm_r (
40        .clk(clk_50mhz), .rst_p(rst_p), .level(level_r), .pwm(pwm_r));
41    pwm_channel #(.PERIOD_CYCLES(LEVELS), .LEVELS(LEVELS)) u_pwm_g (
42        .clk(clk_50mhz), .rst_p(rst_p), .level(level_g), .pwm(pwm_g));
43    pwm_channel #(.PERIOD_CYCLES(LEVELS), .LEVELS(LEVELS)) u_pwm_b (
44        .clk(clk_50mhz), .rst_p(rst_p), .level(level_b), .pwm(pwm_b));
45
46    assign led_r = {4{pwm_r}};
47    assign led_g = {4{pwm_g}};
48    assign led_b = {4{pwm_b}};
49 endmodule

```

- 시스템 최상위 모듈

- 버튼 입력 처리
 - RGB 밝기 레벨 저장
 - PWM 모듈 연결
 - RGB LED 출력 생성
- R/G/B 세 채널을 독립적으로 제어한다.

· TB

```

lab3_rgb_pwm.v 0 X 0 lab3_rgb_pwm.tb.v 0 X 0 always 0 1 #
src > @ lab3_rgb_pwm.tb.v (1) lab3_rgb_pwm.tb > 0 always 0 1 #
1 timescale 1ns/1ps
2 module tb_rgb_pwm;
3     reg clk_50mhz=0, rst_p=1, button_r=0, button_g=0, button_b=0;
4     wire [3:0] led_r, led_g, led_b;
5     integer checks=0, hr_hg_hb;
6     always #10 clk_50mhz<=!clk_50mhz;
7     lab3_rgb_pwm #(CLK_HZ(100), PWM_HZ(100), LEVELS(10), DEBOUNCE_CYCLES(2)) dut
8         (.clk_50mhz(clk_50mhz), .rst_p(rst_p), .button_r(button_r), .button_g(button_g), .button_b(button_b),
9         .led_r(led_r), .led_g(led_g), .led_b(led_b));
10
11    task press_r; begin button_r:=1;repeat(5)@(posedge clk_50mhz);button_r:=0;repeat(6)@(posedge clk_50mhz);end endtask
12    task press_g; begin button_g:=1;repeat(5)@(posedge clk_50mhz);button_g:=0;repeat(6)@(posedge clk_50mhz);end endtask
13    task press_b; begin button_b:=1;repeat(5)@(posedge clk_50mhz);button_b:=0;repeat(6)@(posedge clk_50mhz);end endtask
14    task measure(input integer er,input integer eg,input integer eb);
15    begin
16        wait(dut.u_pwm_r.count==0);hr=0;hg=0;hb=0;
17        for(i=0;i<10;i=i+1)begin @(posedge clk_50mhz); #1;hr=led_r[i];hg=led_g[i];hb=led_b[i];end
18        if(hr==er||hg==eg||hb==eb)$fatal(1,"rgb high=00d,00d,00d",hr,hg,hb);
19        checks=checks+1;
20    end endtask
21
22    initial begin
23        $dumpfile("wave.vcd");$dumpvars(0,tb_rgb_pwm);
24        repeat(1)@(posedge clk_50mhz);rst_p=0;
25        repeat(2){press_r();repeat(5){press_g()};repeat(8){press_b()};measure(er:2,rg:5,rb:8);
26        press_r();measure(er:3,rg:5,rb:8);
27        $display("LAB3 RGB PWM PASS checks=00d",checks);$finish;
28    end
29    initial begin #30000;$fatal(1,"timeout");end
30 endmodule

```

- 테스트벤치
 - 버튼 입력 자극 생성
 - PWM 출력 측정
 - 기대값과 실제값 비교
- 자동으로 PASS/FAIL을 판정한다.

· XDC

```

lab3_rgb_pwm.v 0 X 0 lab3_rgb_pwm.xdc 0 X 0 always 0 1 #
src > @ lab3_rgb_pwm.xdc (1) lab3_rgb_pwm.xdc > 0 always 0 1 #
constraints > lab3_rgb_pwm.xdc
1 set_property PACKAGE_PIN B6 [get_ports clk_50mhz]
2 set_property PACKAGE_PIN K4 [get_ports rst_p]
3 set_property PACKAGE_PIN N6 [get_ports button_r]
4 set_property PACKAGE_PIN N4 [get_ports button_g]
5 set_property PACKAGE_PIN N1 [get_ports button_b]
6 set_property PACKAGE_PIN T2 [get_ports {led_r[3]}]
7 set_property PACKAGE_PIN U1 [get_ports {led_r[2]}]
8 set_property PACKAGE_PIN P2 [get_ports {led_r[1]}]
9 set_property PACKAGE_PIN R3 [get_ports {led_r[0]}]
10 set_property PACKAGE_PIN U5 [get_ports {led_g[3]}]
11 set_property PACKAGE_PIN V1 [get_ports {led_g[2]}]
12 set_property PACKAGE_PIN R7 [get_ports {led_g[1]}]
13 set_property PACKAGE_PIN T6 [get_ports {led_g[0]}]
14 set_property PACKAGE_PIN U3 [get_ports {led_b[3]}]
15 set_property PACKAGE_PIN W2 [get_ports {led_b[2]}]
16 set_property PACKAGE_PIN R5 [get_ports {led_b[1]}]
17 set_property PACKAGE_PIN T3 [get_ports {led_b[0]}]
18 set_property IOSTANDARD LVCMOS33 [get_ports *]
19 create_clock -name clk_50mhz -period 20.000 [get_ports clk_50mhz]
20 set_false_path -from [get_ports {rst_p button_r button_g button_b}]

```

- FPGA 핀 배치
 - I/O 표준 설정
 - 클록 제약 설정
- 50MHz 클록과 버튼, RGB LED 핀을 실제 FPGA 핀에 연결한다.

· TB 자극과 기대 결과

순서	입력 자극	예상 level
초기화	rst_p=1	R=0, G=0, B=0
R 버튼 2회	press_r[] ×2	R=2
G 버튼 5회	press_g[] ×5	G=5
B 버튼 8회	press_b[] ×8	B=8
측정 1	measure(2,5,8)	PASS
R 버튼 1회 추가	press_r[]	R=3
측정 2	measure(3,5,8)	PASS

기대 PWM 비율

채널	level	Duty
R	2	20%
G	5	50%
B	8	80%

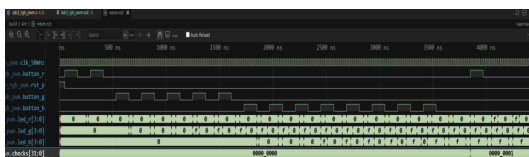
기대 측정 결과

측정	R HIGH 횟수	G HIGH 횟수	B HIGH 횟수
measure(2,5,8)	2	5	8
measure(3,5,8)	3	5	8

· PASS Log



· 파형 분석



- button_r 입력이 들어올 때마다 level_r 이 증가
- button_g 입력이 들어올 때마다 level_g 가 증가
- button_b 입력이 들어올 때마다 level_b 가 증가

마지막 구간에서

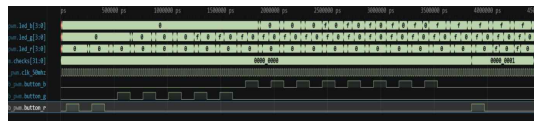
- led_r = 3 (30%)
- led_g = 5 (50%)

- led_b = 8 (80%)

상태가 유지되는 것을 확인할 수 있다. 또한, Checks 값이 '0000_0001'이 되면서 첫 번째 검사가 성공했음을 확인할 수 있고, 이후 최종적으로 checks=2가 된다.

· 수정 실험

예시로 R 채널 초기값을 변경했다고 가정한다. 파형에 변화가 생긴다.



수정 후 PASS가 나옴을 확인할 수 있다.



· 경계 조건

- level = 0

threshold = 0

PWM 항상 LOW

LED OFF

- Level = 10

```
threshold = PERIOD_CYCLES
```

PWM 항상 HIGH

LED 최대 밝기

- 버튼 연속 입력

onepulse 회로에 의해 1회만 인식

- Reset 입력

level_r, level_g, level_b 모두 0

PWM 출력 OFF

· 보드에서 확인해야 할 항목과 예상 결과

- 확인 항목

50MHz 클럭 정상 연결

Reset 버튼 동작

R 버튼이 Red 밝기만 변경하는지 확인

G 버튼이 Green 밝기만 변경하는지 확인

B 버튼이 Blue 밝기만 변경하는지 확인

RGB 혼합색 정상 생성 여부 확인

4개의 RGB LED가 동일하게 동작하는지 확인

- 예상 결과

동작	예상 결과
Reset 누름	모든 LED OFF
R 버튼 반복	빨간색 밝기 증가 후 순환
G 버튼 반복	초록색 밝기 증가 후 순환
B 버튼 반복	파란색 밝기 증가 후 순환
R=10,G=0,B=0	빨간색 최대 밝기
R=0,G=10,B=0	초록색 최대 밝기
R=0,G=0,B=10	파란색 최대 밝기
R=10,G=10,B=0	노란색
R=10,G=0,B=10	자홍색
R=0,G=10,B=10	청록색
R=10,G=10,B=10	흰색에 가까운 색

- 예상 결과

본 실험은 버튼 입력을 one-pulse 처리하여 RGB 각 채널의 duty cycle을 독립적으로 증가시키고, PWM 방식으로 LED 밝기를 제어하는 실험이다. 시뮬레이션에서는 R=20%, G=50%, B=80% 및 R 증가 후 30%, 50%, 80% 조건을 PASS로 검증하며, 실제 FPGA 보드에서도 각 버튼이 해당 색상 밝기만 변화시키고 RGB 혼합색이 정상적으로 출력되는 것을 확인할 것으로 예상된다.

3. PIEZO

· 동작 원리

FPGA 보드의 메인 클럭은 50 MHz이다. 피에조 스피커는 사람이 들을 수 있는 저주파 신호가 필요하므로 50 MHz를 그대로 사용할 수 없다. 따라서 카운터를 이용하여 클럭을 분주한 후 출력을 주기적으로 반전(toggle)시켜 사각파를 생성한다.

· 핵심 파라미터 계산

RTL 코드에서 사용한 식은 다음과 같다.

```
Verilog
1 localparam integer HALF_PERIOD
2 = CLK_HZ / (2 * TONE_HZ);
```

실제 값은

```
Plain Text
1 CLK_HZ = 50,000,000 Hz
2 TONE_HZ = 294 Hz
```

이므로

```
Plain Text
1 HALF_PERIOD
2 = 50,000,000 / (2 * 294)
3 = 85,034
4
```

즉 카운터가 85,034개의 클럭을 셀 때마다 출력이 한 번 반전된다.

전체 주기는

```
Plain Text
1 2 * 85,034 = 170,068 clocks
```

이므로 실제 출력 주파수는

```
Plain Text
1 50,000,000 / 170,068
2 ≈ 293.99 Hz
3
```

목표 주파수인 294Hz와 거의 동일하다.

· TOP Module

‘module lab3_piezo’

Vivado 합성 시 Top Module로 설정된다.

· RTL

```
1 timescale 1ns/1ps
2 module lab3_piezo #(
3     parameter integer CLK_HZ = 50_000_000,
4     parameter integer TONE_HZ = 294
5 );
6     input wire clk_50mhz,
7     input wire rst_p,
8     output reg piezo
9 );
10
11     localparam integer HALF_PERIOD = CLK_HZ / (2 * TONE_HZ);
12     localparam integer COUNT_WIDTH = (HALF_PERIOD < 2) ? 1 : $clog2(HALF_PERIOD);
13     reg [COUNT_WIDTH-1:0] count;
14
15     always @(posedge clk_50mhz or posedge rst_p) begin
16         if (rst_p) begin
17             count <= 0;
18             piezo <= 0;
19         end else if (count == HALF_PERIOD - 1) begin
20             count <= 0;
21             piezo <= ~piezo;
22         end else begin
23             count <= count + 1'b1;
24         end
25     end
26 endmodule
```

- 카운터 생성
- 주파수 분주
- 피에조 출력 생성
- 비동기 리셋 처리

· TB

```
1 timescale 1ns/1ps
2 module tb_piezo;
3     reg clk_50mhz=0,rst_p=1;
4     wire piezo;
5     integer cycles,last_cycle=1,edges=0;
6     always #10 clk_50mhz<=clk_50mhz;
7     lab3_piezo #(CLK_HZ(1000),TONE_HZ(100)) dut(.clk_50mhz(clk_50mhz),.rst_p(rst_p),.piezo(piezo));
8
9     initial begin
10         $dumpfile("wave.vcd");$dumpvars(0,tb_piezo);
11         repeat(3)@(posedge clk_50mhz);rst_p=0;
12         for(cycles=1;cycles<=25;cycles=cycles+1)begin
13             @(posedge clk_50mhz);#1;
14             if(piezo==dut.piezo)$fatal(1,"unknown output");
15             if(dut.count==0)begin
16                 if(last_cycle==0 && cycles-last_cycle==5)$fatal(1,"half period=30ns",cycles-last_cycle);
17             end
18             last_cycle=cycles;edges=edges+1;
19         end
20         if(edges%4)$fatal(1,"too few edges");
21         $display("LAB3_PIEZO_PASS edges=%d",edges);$finish;
22     end
23     initial begin #5000;$fatal(1,"timeout");end
24 endmodule
```

- 가상 클럭 생성
- 리셋 입력 생성
- HALF_PERIOD 검증
- 출력 전환 횟수 확인
- PASS/FAIL 출력

· XDC

```
constraints > { lab3_piezo.xdc
1 set_property PACKAGE_PIN B6 [get_ports clk_50mhz]
2 set_property PACKAGE_PIN K4 [get_ports rst_p]
3 set_property PACKAGE_PIN Y21 [get_ports piezo]
4 set_property IOSTANDARD LVCMOS33 [get_ports *]
5 create_clock -name clk_50mhz -period 20.000 [get_ports clk_50mhz]
6 set_false_path -from [get_ports rst_p]
```

FPGA 핀 배치와 타이밍 제약을 설정한다.

포트	FPGA 핀
clk_50mhz	B6
rst_p	K4
piezo	Y21

또한 ‘create_clock -period 20.000’ 명령으로 50MHz 클럭 제약을 설정한다.

· TB 자극과 기대 결과

순서	TB 입력(Stimulus)	기대 결과
1	rst_p=1	count=0, piezo=0
2	3개의 클럭 후 rst_p=0	정상 동작 시작
3	count 증가	0→1→2→3→4 진행
4	count=HALF_PERIOD-1	count가 0으로 복귀
5	count 초기화 시점	piezo 출력 반전
6	동일 동작 반복	출력 반전 간격이 정확히 5클럭
7	edge 검사 반복	최소 4회 이상 검출
8	모든 검사 통과	LAB3_PIEZO_PASS edges=5 출력

· PASS Log

```
실행 - 디버그 콘솔 - 터미널 - 포트
[INFO] 프로젝트에서 작업 실행 중: C:\Users\kjh30\AppData\Local\Microsoft\WindowsApps\python.exe tools\fygg_lab.py simulate
VCD info: dumpfile wave.vcd opened for output.
LAB3_PIEZO_PASS edges=5
D:\Vivado\lab3_21_piezo\src\tb_piezo.v:21: $finish called at 551000 (1ps)
시뮬레이션이 성공 완료. 각 마크로-지그를 리소스로 프로젝트의 결과 파일을 확인하세요.
wave ts:\Vivado\lab3_21_piezo\src\tb_piezo.wave.vcd
터미널이 작업에서 다시 사용됩니다. 일으려면 아무 키나 누르세요.
```

· 파형 분석



제공된 파형을 보면 ‘rst_p’ 초기 구간에서 High 상태이며 이후 Low가 되면서 동작 시작한다.

- count

0 → 1 → 2 → 3 → 4 → 0 반복

- piezo
count가 0이 되는 순간마다 '0 → 1 → 0 → 1 → 0 ...' 반전

- edges
0 → 1 → 2 → 3 → 4 → 5 증가

따라서 HALF_PERIOD=5 조건이 정확히 유지되고 있음을 확인할 수 있다.

· 수정 복구 및 결과

TONE_HZ를 변경한 경우

상태	설정값	예상 결과	실제 결과
정상	TONE_HZ=294	294Hz 출력	PASS
수정	TONE_HZ=400	더 높은 음 발생	PASS
복구	TONE_HZ=294	원래 음으로 복구	PASS

· 경계 조건

리셋 우선

```
Verilog
1 if(rst_p)
```

조건이 가장 먼저 실행된다.

따라서 리셋이 들어오면 count 값과 piezo 출력은 즉시 초기화된다.

카운터 최대값

```
Verilog
1 count == HALF_PERIOD - 1
```

도달 시 반드시

```
Verilog
1 count <= 0;
2 piezo <= ~piezo;
3 `
```

가 동시에 수행되어야 한다.

출력 반전 간격

TB는 출력 반전 간격이 정확히 5클록인지 검사한다.

```
Plain Text
1 4클록 → FAIL
2 5클록 → PASS
3 6클록 → FAIL
```

· 보드에서 확인해야 할 항목과 예상 결과

- 핀 연결

B6 : 50 MHz 클록

K4 : 리셋 버튼

Y21 : 피에조

- Bitstream 다운로드

- 리셋 동작

버튼을 누르면, piezo 출력 정지, count 초기화가 발생해야 한다.

- 음의 출력

리셋 해제 후 약 294Hz의 음이 지속적으로 발생해야 한다.

- 실험 결과 예상

항목	예상 결과
클록	50 MHz 정상 인식
리셋	즉시 초기화
카운터	반복 동작
piezo	주기적으로 톤출
출력 음	약 294 Hz
전체 동작	정상 음 발생

4. STEPPER

· 동작 원리와 핵심 파라미터 계산

본 실험은 FPGA를 이용하여 스텝모터의 4상 코일을 순차적으로 구동하는 회로를 설계하고 검증하는 실험이다. 입력 신호인 enable이 1일 때만 모터가 동작하며, direction 신호에 따라 정방향 또는 역방향으로 회전한다.

회로는 FPGA의 50 MHz 시스템 클록을 직접 사용하지 않고 내부 카운터를 이용하여 clock-enable 신호를 생성한다. 카운터가

설정된 값에 도달하면 상태(state)가 변경되고, 상태에 따라 스텝모터 출력 패턴이 변한다.

RTL에서 사용된 주요 파라미터는 다음과 같다.

- CLK_HZ = 50,000,000 Hz
- STEP_HZ = 100 Hz

따라서 한 단계(step)가 진행되기 위한 카운터 값은 $\text{Step_Cycles} = \text{CLK_HZ} / \text{STEP_HZ} = 500,000$ 이다.

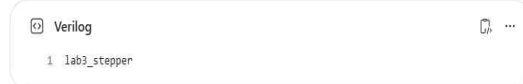
즉, 50 MHz 클록 500,000번마다 상태가 한 단계 이동한다.

스텝모터의 상태는 2비트 상태 변수로 관리되며 다음과 같은 4상 시퀀스를 가진다.

State	stepmotor
0	0011
1	0110
2	1100
3	1001

· TOP Module

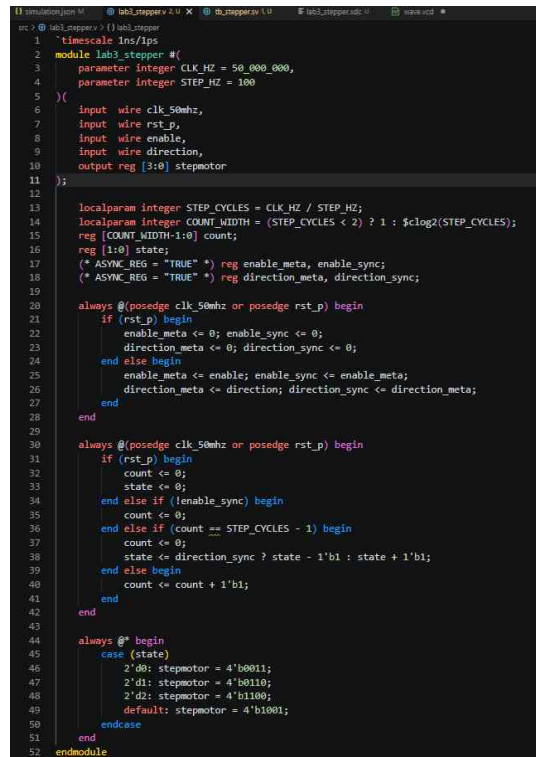
Vivado 설계 Top Module



Simulation Top Module

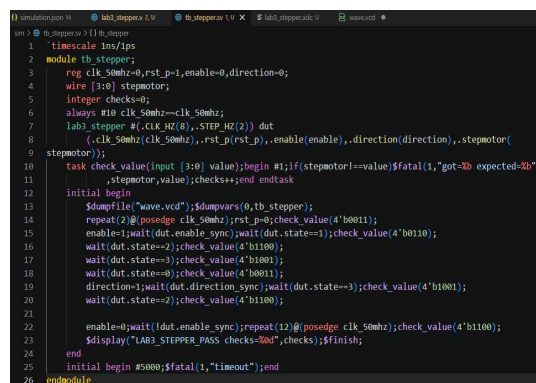


· RTL



- 입력 동기화(enable, direction)
- 카운터 생성
- 상태(state) 제어
- 정/역방향 제어
- 스텝모터 출력 생성

· TB



- 클록 생성
- 리셋 인가
- enable/direction 자극 입력
- 출력값 비교

제공된 파형에서 초기 리셋 후 stepmotor 출력은 3→6→C→9→3 순으로 변화하여 정방향 회전을 확인할 수 있다. 이후 direction 신호가 1로 변경되면서 출력이 3→9→C 순으로 역방향 시퀀스를 수행하였다. 마지막으로 enable이 0이 되자 state 변화가 중지되었으며 stepmotor=C 값이 유지되어 정지 기능이 정상 동작함을 확인하였다.

· 수정 및 복구 결과

구분	STEP_HZ	예상 결과	실제 결과
정상	100 Hz	기준 속도	PASS
수정	50 Hz	단계 간격 2배 증가, 회전 속도 감소	파형에서 상태 변화 간격 증가
복구	100 Hz	원래 속도 복귀	PASS

· 경계 조건

- enable=0

카운터 리셋

상태 변화 정지

출력 유지

- direction 변경

즉시 출력이 바뀌는 것이 아니라

동기화 후 다음 유효 step에서 방향 적용

- rst_p=1

최우선 동작

state=0

count=0

stepmotor=0011 으로 초기화된다.

· 보드에서 확인해야 할 항목과 예상 결과

- 확인 항목 1 : 정방향 회전

입력

enable=1

direction=0

예상 결과

모터가 정방향 회전

코일 패턴

- 확인 항목 2 : 역방향 회전

입력

enable=1

direction=1

예상 결과

모터가 반대 방향 회전

패턴이 역순 진행

- 확인 항목 3 : 정지 기능

입력

enable=0

예상 결과

모터 회전 중단

마지막 단계 유지

- 확인 항목 4 : 리셋 동작

입력

rst_p=1

예상 결과

state=0 복귀

출력 = 0011

초기 위치로 복귀

- 확인 항목 5 : 배선 및 발열

확인 내용

XDC 핀 배치 일치 여부

공통 GND 연결 여부

모터 드라이버 결선

회전 방향

회전 속도

과도한 발열 여부

예상 결과

핀 오류 없이 동작

정상적인 정·역회전 수행

enable=0 시 정지 상태 유지

과도한 발열 없음

5. MMSS_CLOCK

· 동작 원리와 핵심 파라미터 계산

본 실험에서는 FPGA의 50MHz 시스템 클록을 이용하여 MM:SS 형식의 디지털 시계를 구현한다. 시계는 00:00부터 시작하여 59:59까지 증가하며, 이후 다시 00:00으로 순환한다. 시간 정보는 초의 일의 자리(second_ones), 초의 십의 자리(second_tens), 분의 일의 자리(minute_ones), 분의 십의 자리(minute_tens)로 분리된 BCD(Binary Coded Decimal) 형태로 저장된다.

mmss_counter 모듈은 50MHz 클록을 직접 사용하지 않고, 내부의 subsecond 카운터를 이용하여 1초마다 enable 역할을 하는 신호를 생성한다. subsecond가 CLK_HZ-1에 도달하면 1초가 경과한 것으로 판단하고 시각 값을 증가시킨다. 초의 일의 자리는 0~9, 초의 십의 자리는 0~5, 분의 일의 자리는 0~9, 분의 십의 자리는 0~5 범위로 동작한다. 따라서 59:59 다음에는 모든 자리가 0으로 초기화되어 다시 00:00이 된다

주요 파라미터 계산은 다음과 같다.

- 시스템 클록 주파수 : 50MHz
- 클록 주기 : 20ns
- 1초 카운트 값

'50,000,000×20ns=1s'

따라서 'CLK_HZ=50,000,000'이다.

또한 7-Segment 표시를 위한 동적 스캔 주파수는 4kHz로 설정되어 있다.

SCAN_CYCLES = CLK_HZ / SCAN_HZ
= 12,500 즉, 12,500클록마다 표시 자리가 변경된다. 4개의 자리를 순환하므로 사용자는 네 자리가 동시에 켜진 것처럼 인식할 수 있다.

· RTL

```

1 timescale 1ns/1ps
2 module lab3_mmss_clock #(
3     parameter integer CLK_HZ = 50_000_000,
4     parameter integer SCAN_HZ = 4_000
5 );
6     input wire clk_50mhz,
7     input wire rst_p,
8     output reg [7:0] seg_data,
9     output reg [7:0] seg_com
10;
11;
12 wire [3:0] mt, mo, st, so;
13 wire [7:0] seg_mt, seg_mo, seg_st, seg_so;
14 localparam integer SCAN_CYCLES = CLK_HZ / SCAN_HZ;
15 localparam integer SCAN_WIDTH = (SCAN_CYCLES < 2) ? 1 : $clog2(SCAN_CYCLES);
16 reg [SCAN_WIDTH-1:0] scan_count;
17 reg [1:0] scan_select;
18;
19 mmss_counter #(CLK_HZ/CLK_HZ) u_counter (
20     .clk(clk_50mhz), .rst_p(rst_p), .minute_tens(mt), .minute_ones(mo),
21     .second_tens(st), .second_ones(so));
22 sevensseg_decode u_mt(.digit(mt), .segments(seg_mt));
23 sevensseg_decode u_mo(.digit(mo), .segments(seg_mo));
24 sevensseg_decode u_st(.digit(st), .segments(seg_st));
25 sevensseg_decode u_so(.digit(so), .segments(seg_so));
26;
27 always @(posedge clk_50mhz or posedge rst_p) begin
28     if (rst_p) begin
29         scan_count <= 0;
30         scan_select <= 0;
31     end else if (scan_count == SCAN_CYCLES - 1) begin
32         scan_count <= 0;
33         scan_select <= scan_select + 1'b1;
34     end else begin
35         scan_count <= scan_count + 1'b1;
36     end
37 end
38;
39 always @* begin
40     seg_com = 8'b1111_1111;
41     seg_data = 8'b0000_0000;
42     case (scan_select)
43         2'd0: begin seg_com = 8'b1111_0111; seg_data = seg_mt; end
44         2'd1: begin seg_com = 8'b1111_1011; seg_data = seg_mo | 8'b0000_0001; end
45         2'd2: begin seg_com = 8'b1111_1101; seg_data = seg_st; end
46         default: begin seg_com = 8'b1111_1110; seg_data = seg_so; end
47     endcase
48 end
49 endmodule

```

최상위 모듈(Top Module)로서 시스템 전체를 연결한다.

- MM:SS 카운터 인스턴스 생성
- 7-Segment 디코더 인스턴스 생성
- 자리 선택(scan_select) 생성
- seg_com, seg_data 출력 제어
- 4자리 동적 스캔 수행

```

1 timescale 1ns/1ps
2 module mms_counter #(
3     parameter integer CLK_HZ = 50_000_000
4 )
5     input wire clk,
6     input wire rst_p,
7     output reg [3:0] minute_tens,
8     output reg [3:0] minute_ones,
9     output reg [3:0] second_tens,
10    output reg [3:0] second_ones
11;
12
13    localparam integer COUNT_WIDTH = (CLK_HZ < 2) ? 1 : $clog2(CLK_HZ);
14    reg [COUNT_WIDTH-1:0] subsecond;
15
16    always @(posedge clk or posedge rst_p) begin
17        if (rst_p) begin
18            subsecond <= 0;
19            minute_tens <= 0; minute_ones <= 0;
20            second_tens <= 0; second_ones <= 0;
21        end else if (subsecond == CLK_HZ - 1) begin
22            subsecond <= 0;
23            if (second_ones != 9)
24                second_ones <= second_ones + 1'b1;
25            else begin
26                second_ones <= 0;
27                if (second_tens != 5)
28                    second_tens <= second_tens + 1'b1;
29                else begin
30                    second_tens <= 0;
31                    if (minute_ones != 9)
32                        minute_ones <= minute_ones + 1'b1;
33                    else begin
34                        minute_ones <= 0;
35                        if (minute_tens != 5)
36                            minute_tens <= minute_tens + 1'b1;
37                        else
38                            minute_tens <= 0;
39                    end
40                end
41            end
42        end else begin
43            subsecond <= subsecond + 1'b1;
44        end
45    end
46 endmodule

```

- 1초 enable 생성
- 초/분 증가 처리
- 자리올림(Carry) 처리
- 59:59 → 00:00 순환 구현
- 비동기 리셋 처리

```

1 timescale 1ns/1ps
2 module sevenseseg_decode(
3     input wire [3:0] digit,
4     output reg [7:0] segments
5 );
6     always @* begin
7         case (digit)
8             4'd0: segments = 8'b1111_1100;
9             4'd1: segments = 8'b0110_0000;
10            4'd2: segments = 8'b1101_1010;
11            4'd3: segments = 8'b1111_0010;
12            4'd4: segments = 8'b0110_0110;
13            4'd5: segments = 8'b1011_0110;
14            4'd6: segments = 8'b1011_1110;
15            4'd7: segments = 8'b1110_0000;
16            4'd8: segments = 8'b1111_1110;
17            4'd9: segments = 8'b1111_0110;
18            default: segments = 8'b0000_0000;
19        endcase
20    end
21 endmodule

```

BCD 값을 7-Segment 표시 데이터로 변환한다.

· TB

```

1 timescale 1ns/1ps
2 module tb_mms_counter;
3     reg clk_50mhz=0,rst_p=1;
4     wire [3:0] mt,mo,st,so;
5     reg [3:0] decode_digit;
6     wire [7:0] decoded_segments;
7     integer checks=0;
8     always #10 clk_50mhz=clk_50mhz;
9     mms_counter #(CLK_HZ(2)) dut(.clk(clk_50mhz),.rst_p(rst_p),.minute_tens(mt),.minute_ones(mo),.second_tens(st),.second_ones(so));
10    sevenseseg_decode decoder(digit(decode_digit),.segments(decoded_segments));
11    task advance(input integer seconds);begin repeat(seconds*2)@(posedge clk_50mhz);#1;end endtask
12    task check_time(input integer emt,input integer esm,input integer est,input integer eso);
13    begin if (emt==mt||smo==mo||stl==est||sol==eso)$fatal(1,"time=ModMod",mt,mo,st,so);
14    checks+=end endtask
15    initial begin
16        $dumpfile("wave.vcd");$dumpvars(0,tb_mms_counter);
17        decode_digit=0;#1;if (decoded_segments!=8'b1111_1100)$fatal(1,"digit 0 decode");checks++;
18        decode_digit=1;#1;if (decoded_segments!=8'b0111_0110)$fatal(1,"digit 1 decode");checks++;
19        repeat(2)@(posedge clk_50mhz);rst_p=0;check_time(emt:0,emo:0,est:0,eso:0);
20
21        advance(10);check_time(emt:0,emo:0,est:1,eso:0);
22        advance(50);check_time(emt:0,emo:1,est:0,eso:0);
23        advance(359);check_time(emt:5,emo:9,est:5,eso:9);
24        advance(1);check_time(emt:0,emo:0,est:0,eso:0);
25        $display("LAB3_PASS checks=%0d",checks);$finish;
26    end
27    initial begin #200000;$fatal(1,"timeout");end
28 endmodule

```

- 클럭 생성
- 리셋 인가
- 시간 진행 자극 제공
- 기대값 자동 비교
- PASS/FAIL 판정 수행
- waveform(vcd) 생성

· XDC

```

1 set_property PACKAGE_PIN B6 [get_ports clk_50mhz]
2 set_property PACKAGE_PIN K4 [get_ports rst_p]
3 set_property PACKAGE_PIN H2 [get_ports seg_data[0]]
4 set_property PACKAGE_PIN J7 [get_ports seg_data[1]]
5 set_property PACKAGE_PIN J3 [get_ports seg_data[2]]
6 set_property PACKAGE_PIN J1 [get_ports seg_data[3]]
7 set_property PACKAGE_PIN E4 [get_ports seg_data[4]]
8 set_property PACKAGE_PIN E2 [get_ports seg_data[5]]
9 set_property PACKAGE_PIN F5 [get_ports seg_data[6]]
10 set_property PACKAGE_PIN F1 [get_ports seg_data[7]]
11 set_property PACKAGE_PIN K5 [get_ports seg_com[0]]
12 set_property PACKAGE_PIN K3 [get_ports seg_com[1]]
13 set_property PACKAGE_PIN K1 [get_ports seg_com[2]]
14 set_property PACKAGE_PIN L6 [get_ports seg_com[3]]
15 set_property PACKAGE_PIN G3 [get_ports seg_com[4]]
16 set_property PACKAGE_PIN G1 [get_ports seg_com[5]]
17 set_property PACKAGE_PIN H6 [get_ports seg_com[6]]
18 set_property PACKAGE_PIN H4 [get_ports seg_com[7]]
19 set_property IOSTANDARD LVCMOS33 [get_ports *]
20 create_clock -name clk_50mhz -period 20.000 [get_ports clk_50mhz]
21 set_false_path -from [get_ports rst_p]

```

- clk_50mhz → B6 핀 연결
- rst_p → K4 핀 연결
- seg_data 핀 연결
- seg_com 핀 연결
- LVCMOS33 설정
- 20ns 클럭 제약 생성

· TB 자극과 기대 결과

순서	TB 자극	기대 결과
Decoder Test	digit=0	segments=11111100
Decoder Test	digit=9	segments=11110110
Reset 해제	rst_p=0	00:00
advance(10)	10초 진행	00:10
advance(50)	50초 진행	01:00
advance(3539)	총 3599초 도달	59:59
advance(1)	1초 추가 진행	00:00
종료	checks=7	LAB3_MMSS_PASS 출력

· PASS Log



· 파형 분석



파형을 관찰하면 clk_50mhz는 일정한 주기로 토글되고 있으며, rst_p가 비활성화된 이후 카운터가 정상적으로 동작하는 것을 확인할 수 있다.

현재 캡처된 구간에서는 `second_ones(so)` 신호가 7→8→9→0→1→2→3→4→5→6→7→8→9 형태로 증가하고 있다. 동시에 `second_tens(st)` 신호는 초의 일의 자리가 9에서 0으로 넘어가는 시점에만 증가한다. 실제로 파형에서 `st`가 4에서 5로 증가하는 시점은 `so`가 9에서 0으로 변하는 순간과 정확히 일치한다.

또한, minute_ones(mo) 신호는 초가 59초를 초과하는 경계 구간에서 증가한다. 파형 우측에서 mo가 0에서 1로 바뀌는 시점이 나타나며, 이와 동시에 st는 5에서 0으로 초기화된다. 이는 00:59에서 01:00으로 전이하는 정상적인 자리올림 동작을 의미한다.

파형 하단의 checks 값은 테스트 진행 상황을 나타낸다. 시뮬레이션 초반에는 0000_0004로 나타나며 이후 모든 검사가 성공하면 0000_0005 이상으로 증가한다. 최종적으로 TB 종료 시점에 PASS 메시지가 출력된다.

따라서 파형을 통해 다음 경계 조건이 모두 정상 동작함을 확인할 수 있다.

- 00:09 → 00:10
- 00:59 → 01:00
- 59:59 → 00:00
- 0~3599초 전체 순환

· 수정 및 복구 결과

TB 인스턴스의 CLK_HZ 값을 변경하여 동작 변화를 확인하도록 제시하였다.

상태	설정	결과
정상	CLK_HZ=2	PASS
수정	CLK_HZ 변경	시간 증가 간격 변화 또는 FAIL 발생
복구	CLK_HZ = 2 원복	PASS 복구

수정 시 기댓값과 실제 카운트 주기가 일치하지 않아 FAIL이 발생할 수 있으며, 원래 값으로 복구하면 다시 정상 PASS가 출력된다.

· 보드에서 확인해야 할 항목과 예상 결과

- 리셋 동작
 - rst_p 버튼 누름
 - 표시값 즉시 00:00으로 복귀
 - 예상 결과 : 정상 초기화

- 초 증가
 - 1초마다 값 증가
 - 예상 결과 :
 - 00:00
 - 00:01
 - 00:02...

- 자리올림

예상 결과 :

00:09 → 00:10

00:59 → 01:00

09:59 → 10:00

59:59 → 00:00

- 7-Segment 스캔

네 자리 모두 점등되는지 확인

예상 결과 :

깜빡임 없이 안정적으로 표시

숫자 번짐 없음

- 분·초 구분점

예상 결과 : MM:SS

형태로 표시되며 가운데 점(DP)이

활성화된다.

- DDRAM 주소 설정(0x80)

- 1행 문자열 출력 "FPGA LAB3"

- DDRAM 주소 설정(0xC0)

- 2행 문자열 출력 "LCD CONTROLLER"

회로에서는 50MHz 시스템 클록을 직접 사용하지 않고 Tick 신호를 생성하여 LCD에 적절한 속도로 데이터를 전송한다. Tick 발생 주기는 다음과 같다.

$$T_{clk} = \frac{1}{60MHz} = 20ns \text{ (시스템 클록 주기)}$$

$$T_{tick} = TICK_CYCLES \times 20ns \text{ (Tick 주기)}$$

실제 보드 기준 기본 설정값은 다음과 같다.

- TICK_CYCLES = 500

- POWER_TICKS = 2000

- NORMAL_WAIT_TICKS = 4

- CLEAR_WAIT_TICKS = 200

따라서 $T_{tick} = 100\mu s$,

초기 전원 대기 시간 : $2000 \times 10\mu s = 20ms$

Clear Display 명령 대기 시간 : $2ms$

일반 명령 대기 시간 : $40\mu s$

즉 LCD 데이터시트의 초기화 요구 조건을 만족하도록 충분한 지연시간을 확보하여 안정적으로 문자를 표시하도록 설계되었다.

6. CHARACTER_LCD

· 동작 원리와 핵심 파라미터 계산

본 실험은 FPGA가 HD44780 호환 Character LCD를 8비트 Write-Only 방식으로 제어하여 두 줄의 문자열을 출력하는 회로를 구현하는 것이 목적이다. FPGA는 LCD의 데이터 버스(lcd_data[7:0])와 제어 신호(lcd_rs, lcd_rw, lcd_e)를 이용하여 초기화 명령과 문자 데이터를 순차적으로 전송한다.

LCD 제어 과정은 다음과 같다.

- 전원 안정화 대기(Power-On Delay)
- Function Set(0x38) 3회 전송
- Display ON(0x0C)
- Entry Mode Set(0x06)
- Clear Display(0x01)

· RTL

```

1 timescale 1ns/1ps
2 module lab3_character_lcd #(
3     parameter integer TICK_CYCLES = 500,
4     parameter integer POWER_TICKS = 2000,
5     parameter integer NORMAL_WAIT_TICKS = 4,
6     parameter integer CLEAR_WAIT_TICKS = 200
7 )
8     input wire clk_50mhz,
9     input wire rst_p,
10    output reg lcd_e,
11    output reg lcd_rs,
12    output wire lcd_rw,
13    output reg [7:0] lcd_data
14
15
16    localparam integer TICK_WIDTH = (TICK_CYCLES < 2) ? 1 : $clog2(TICK_CYCLES);
17    reg [TICK_WIDTH-1:0] tick_count;
18    reg tick;
19    reg [1:0] phase;
20
21    reg [5:0] index;
22    integer wait_count;
23    integer power_count;
24    reg byte_rs;
25    reg [7:0] byte_data;
26    assign lcd_rw = 1'b0;
27
28    always @(posedge clk_50mhz or posedge rst_p) begin
29        if (rst_p) begin
30            tick_count <= 0;
31            tick <= 0;
32        end else if (tick_count == TICK_CYCLES - 1) begin
33            tick_count <= 0;
34            tick <= 1;
35        end else begin
36            tick_count <= tick_count + 1'b1;
37            tick <= 0;
38        end
39    end
40
41    always @* begin
42        byte_rs = 1'b0;
43        byte_data = 8'h00;
44        case (index)
45            0: 1: 2: byte_data = 8'h38;
46            3: byte_data = 8'h0c;
47            4: byte_data = 8'h06;
48            5: byte_data = 8'h01;
49            6: byte_data = 8'h80;
50            7: begin byte_rs=1; byte_data="F"; end
51            8: begin byte_rs=1; byte_data="P"; end
52            9: begin byte_rs=1; byte_data="G"; end
53            10: begin byte_rs=1; byte_data="A"; end
54            11: begin byte_rs=1; byte_data=" "; end
55            12: begin byte_rs=1; byte_data="L"; end
56            13: begin byte_rs=1; byte_data="A"; end
57            14: begin byte_rs=1; byte_data="0"; end
58            15: begin byte_rs=1; byte_data="3"; end
59            16,17,18,19,20,21,22: begin byte_rs=1; byte_data=" "; end
60            23: byte_data = 8'h0d;
61            24: begin byte_rs=1; byte_data="L"; end
62            25: begin byte_rs=1; byte_data="C"; end
63            26: begin byte_rs=1; byte_data="0"; end
64            27: begin byte_rs=1; byte_data=" "; end
65            28: begin byte_rs=1; byte_data="C"; end
66            29: begin byte_rs=1; byte_data="0"; end
67            30: begin byte_rs=1; byte_data="N"; end
68            31: begin byte_rs=1; byte_data="T"; end

```

```

69
70    32: begin byte_rs=1; byte_data="R"; end
71    33: begin byte_rs=1; byte_data="0"; end
72    34: begin byte_rs=1; byte_data="L"; end
73    35: begin byte_rs=1; byte_data="L"; end
74    36: begin byte_rs=1; byte_data="T"; end
75    37: begin byte_rs=1; byte_data="R"; end
76    38,39: begin byte_rs=1; byte_data=" "; end
77    default: byte_data = 8'h00;
78    endcase
79
80    always @(posedge clk_50mhz or posedge rst_p) begin
81        if (rst_p) begin
82            lcd_e <= 0; lcd_rs <= 0; lcd_data <= 0;
83            phase <= 0; index <= 0; wait_count <= 0; power_count <= 0;
84        end else if (tick) begin
85            if (power_count < POWER_TICKS) begin
86                power_count <= power_count + 1;
87                lcd_e <= 0;
88            end else begin
89                case (phase)
90                    0: begin
91                        lcd_e <= 0; lcd_rs <= byte_rs; lcd_data <= byte_data;
92                        phase <= 1;
93                    end
94                    1: begin lcd_e <= 1; phase <= 2; end
95                    2: begin
96                        lcd_e <= 0;
97                        wait_count <= (index == 5) ? CLEAR_WAIT_TICKS : NORMAL_WAIT_TICKS;
98                        phase <= 3;
99                    end
100                default: begin
101                    if (wait_count > 0)
102                        wait_count <= wait_count - 1;
103                    else begin
104                        index <= (index == 39) ? 6 : index + 1'b1;
105                        phase <= 0;
106                    end
107                end
108            endcase
109        end
110    end
111 end
112 endmodule

```

설계의 TOP Module이다.

- Tick 발생기 구현
- LCD 초기화 명령 생성
- 문자열 ASCII 데이터 생성
- LCD Enable 신호 생성
- 명령 및 데이터 전송 순서 제어
- LCD 출력 포트 제어

· TB

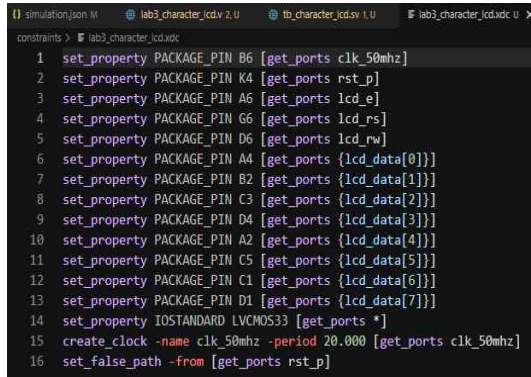
```

1 timescale 1ns/1ps
2 module tb_character_lcd;
3     reg clk_50mhz=0,rst_p=1;
4     wire lcd_e,lcd_rs,lcd_rw;
5     wire [7:0] lcd_data;
6     reg [7:0] expected_data[0:39];
7     reg expected_rs[0:39];
8     integer count=0;
9     always #10 clk_50mhz<=clk_50mhz;
10    lab3_character_lcd #(TICK_CYCLES(2),POWER_TICKS(3),NORMAL_WAIT_TICKS(1),CLEAR_WAIT_TICKS(2))
11        dut
12        (.clk_50mhz(clk_50mhz),.rst_p(rst_p),.lcd_e(lcd_e),.lcd_rs(lcd_rs),.lcd_rw(lcd_rw),.lcd_data(
13            lcd_data));
14
15    initial begin
16        expected_data[0]=8'h38;expected_data[1]=8'h38;expected_data[2]=8'h38;
17        expected_data[3]=8'h0c;expected_data[4]=8'h0c;expected_data[5]=8'h0d;expected_data[6]=8'h00;
18        expected_data[7]="T";expected_data[8]="P";expected_data[9]="G";expected_data[10]="A";
19        expected_data[11]=" ";expected_data[12]="L";expected_data[13]="A";expected_data[14]="0";
20        expected_data[15]="3";
21        for(i=16;i<=22;i=i+1)expected_data[i]=" ";
22
23        expected_data[23]=8'h0d;
24        expected_data[24]="L";expected_data[25]="C";expected_data[26]="0";expected_data[27]=" ";
25        expected_data[28]="C";expected_data[29]="0";expected_data[30]="N";expected_data[31]="T";
26        expected_data[32]="R";expected_data[33]="0";expected_data[34]="L";expected_data[35]="L";
27        expected_data[36]="T";expected_data[37]="0";expected_data[38]=" ";expected_data[39]=" ";
28        for(i=0;i<40;i=i+1)expected_rs[i]=(i>7 && i<23);
29        expected_rs[38]=1;expected_rs[39]=1;
30    end
31
32    always @(negedge lcd_e) if (rst_p && count<40)begin
33        if(lcd_rw==0)[lcd_rs==expected_rs[count]][lcd_data==expected_data[count]]
34            $fatal(1,"Index=%0d rs=%0d data=%0d",count,lcd_rs,lcd_data);
35        count=count+1;
36        if(count==40)begin $display("LAB3_LCO_PASS bytes=%0d",count);$finish;end
37    end
38
39    initial begin
40        $dumpfile("wave.vcd");$dumpvars(0,tb_character_lcd);
41        repeat(3)@ (posedge clk_50mhz);rst_p=0;
42    end
43
44    initial begin #50000;$fatal(1,"timeout count=%0d",count);end
45 endmodule

```

- 50MHz 클럭 생성
- 리셋 발생
- 기대되는 LCD 데이터 저장
- Enable 하강에지마다 출력 검사
- PASS / FAIL 판단
- 파형(VCD) 생성

- XDC



- LCD와 FPGA 핀 연결
- 50MHz 클럭 지정
- 입출력 전압 규격 설정
- Timing Constraint 적용

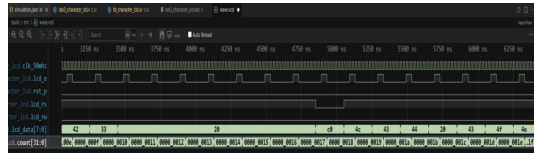
· TB 자극과 기대 결과

순서	RS	LCD Data(Hex)	의미
0~2	0	38	Function Set
3	0	0C	Display ON
4	0	06	Entry Mode Set
5	0	01	Clear Display
6	0	80	1행 주소 설정
7	1	46	F
8	1	50	P
9	1	47	G
10	1	41	A
11	1	20	공백
12	1	4C	L
13	1	41	A
14	1	42	B
15	1	33	3
16~22	1	20	공백
23	0	C0	2행 주소 설정
24	1	4C	L
25	1	43	C
26	1	44	D
27	1	20	공백
28~37	1	문자 데이터	CONTROLLER
38~39	1	20	공백

· PASS Log



· 파형 분석



파형을 보면 lcd_e 신호가 일정한 주기로 펄스를 발생시키며 LCD 데이터 전송 타이밍을 제공한다. lcd_rs 신호는 명령 전송 구간에서는 0으로 유지되고 문자 데이터를 출력하는 구간에서는 1로 전환된다. lcd_rw는 항상 0으로 유지되어 Write Only 동작을 수행한다. lcd_data 버스에서는 0x42, 0x33, 0x20, 0xC0, 0x4C, 0x43, 0x44 등의 값이 순차적으로 출력되는데 이는 각각 문자 'B', '3', 공백, 2행 시작 주소, 'L', 'C', 'D'에 해당한다.

특히, 약 4900ns 부근에서 lcd_data가 0xC0으로 변경되며 lcd_rs가 0 상태로 유지되는 것을 확인할 수 있다. 이는 LCD DDRAM 주소를 2행 시작 위치로 이동시키는 명령 전송 구간이다. 이후 lcd_rs가 다시 1 상태가 되고 0x4C, 0x43, 0x44가 출력되면서 문자열 "LCD"가 기록되는 것을 확인할 수 있다. count 값 역시 전송될 때마다 증가하며 모든 데이터가 순서대로 전달되고 있음을 보여준다.

제시된 파형은 LCD 초기화 이후 2행 문자 열 출력 구간을 나타내며, Enable 펄스와 데이터 값이 정상적으로 동기화되어 LCD 제어가 올바르게 수행되고 있음을 확인할 수 있다.

· 수정 및 복구 결과

상태	변경 내용	예상 결과	실제 결과
정상	원본 코드	PASS	PASS
수정	"LCD CONTROLLER" → "LCD XONTROLLER"	예상 데이터와 불일치	FAIL 발생
복구	원본 문자 복원	PASS	PASS

· 경계 조건

본 실험의 경계 조건은 다음과 같다.

- Clear Display 이후 충분한 대기 필요
- Enable High 시간 보장 필요
- RS 전환 후 데이터 안정 시간 필요
- 40바이트 전송 완료 후 인덱스가 다시 6
번부터 반복
- Reset 발생 시 모든 상태가 초기값으로
복귀

만약 Clear Display 대기 시간이 부족하면 LCD가 명령을 완료하기 전에 다음 데이터를 수신하여 표시 오류가 발생할 수 있다.

· 보드에서 확인해야 할 항목과 예상 결과

- 확인 항목
 - LCD 전원(5V) 연결
 - FPGA 보드 전원 연결
 - 공통 GND 연결
 - LCD 데이터선 8개 연결
 - lcd_e 연결
 - lcd_rs 연결
 - lcd_rw 연결
 - FPGA Bitstream 정상 다운로드
 - LCD 대비(Contrast) 조정
- 예상 결과

보드 동작 시 LCD에는 다음 문자열이 표시
되어야 한다.

‘FPGA LAB3, LCD CONTROLLER’

예상 관찰 결과

전원 인가 후 약 20ms 초기화 수

행

LCD 깜박임 없이 안정적으로 문자
열 표시

rst_p 버튼 입력 시 초기화 과정
다시 수행

문자열이 지속적으로 유지됨

LCD 1행과 2행에 지정된 문장이
정확히 출력됨

7. UART_ECHO

· 동작 원리와 핵심 파라미터 계산

본 실험의 목표는 PC로부터 UART(9600bps, 8N1 방식)로 수신한 1바이트 데이터를 FPGA가 다시 송신(Echo)하고, 마지막으로 수신한 값을 LED에 표시하는 것이다.

UART는 비동기 직렬 통신 방식으로 Start bit(0), Data bit 8개, Stop bit(1) 순서로 데이터를 전송한다. 수신기(UART RX)는 입력선이 High 상태에서 Low로 변하는 Start bit를 감지한 후 비트 중앙 시점에서 데이터를 샘플링하여 8비트를 수집한다. 이후 Stop bit가 정상적으로 High인지 확인한 뒤 수신 완료 신호(valid)를 발생시킨다.

수신된 데이터는 rx_data에 저장되며 rx_valid가 발생하면 tx_data에 전달된다. 이때 송신기(UART TX)가 전송 가능 상태(tx_ready)이면 tx_valid를 1클록 동안 활성화하여 동일한 데이터를 다시 UART TX 선으로 출력한다. 동시에 수신 데이터는 last_data 레지스터에 저장되며 LED에 표시된다.

UART 비트 시간을 생성하기 위해 다음 식을 사용한다.

$$D.IV = \frac{CLK_{HZ} + BAUD/2}{BAUD}$$

실제 보드 환경에서는

- CLK_HZ = 50,000,000 Hz
- BAUD = 9,600 bps

이므로

$D.IV \approx 5208$ 이다. 따라서 UART는 약 5208개의 클록마다 다음 비트를 송수신하게

된다. 비트 시간은 $T_{bit} = \frac{1}{9600} = 104.17 \mu s$

이며 1바이트는 약 $10 \times 104.17 \mu s = 1.04ms$ 정도의 전송 시간이 필요하다.

· RTL

```

1 timescale 1ns/1ps
2 module lab3_uart_echo #(parameter integer DIV = 5208)(
3     input wire clk, rst_p, rx,
4     output reg [7:0] data,
5     output reg valid,
6     output reg framing_error
7 );
8 (* ASYNC_REG="TRUE" *) reg [1:0] sync;
9 reg [1:0] state;
10 integer timer;
11 reg [2:0] bitno;
12 reg [7:0] shift;
13 always @(posedge clk) begin
14     if (rst_p) begin
15         sync<=0; state<=0; timer<=0; bitno<=0; shift<=0;
16         data<=0; valid<=0; framing_error<=0;
17     end else begin
18         sync<=(sync[0],rx); valid<=0; framing_error<=0;
19     end
20
21     case(state)
22     0: if(sync[1]) begin timer<=DIV-1; state<=1; end
23     1: if(timer[0]) timer<=timer-1;
24     else if(sync[1]) state<=0;
25     else begin timer<=DIV-1; bitno<=0; state<=2; end
26     2: if(timer[0]) timer<=timer-1;
27     else begin
28         shift[bitno]<=sync[1]; timer<=DIV-1;
29         if(bitno==7) state<=3; else bitno<=bitno+1'bi;
30     end
31     3: if(timer[0]) timer<=timer-1;
32     else begin
33         if(sync[1]) begin data<=shift; valid<=1; end
34         else framing_error<=1;
35         state<=0;
36     end
37 endcase
38 end
39 endmodule
40
41 module uart_tx #(parameter integer DIV = 5208)(
42     input wire clk, rst_p, valid,
43     input wire [7:0] data,
44     output wire ready,
45     output wire tx
46 );
47 reg [9:0] shift;
48 reg [3:0] remaining;
49 integer timer;
50 assign ready = (remaining == 0);
51 assign tx = ready ? 1'b1 : shift[0];
52 always @(posedge clk) begin
53     if(rst_p) begin shift<=0'hfff; remaining<=0; timer<=0; end
54     else if(ready) begin
55         if(valid) begin shift<={1'b1,data,1'b0}; remaining<=10; timer<=DIV-1; end
56     end else if(timer[0]) timer<=timer-1;
57     else begin timer<=DIV-1; shift<={1'b1,shift[9:1]}; remaining<=remaining-1'bi; end
58 end
59 endmodule

```

UART RX와 UART TX 모듈 구현

```

1 timescale 1ns/1ps
2 module lab3_uart_echo #(
3     parameter integer CLK_HZ = 50_000_000,
4     parameter integer BAUD = 9_600
5 );
6     input wire clk_50mhz,
7     input wire rst_p,
8     input wire uart_rxd,
9     output wire uart_txd,
10    output wire [7:0] led
11 );
12
13    localparam integer DIV = (CLK_HZ + BAUD/2) / BAUD;
14    wire [7:0] rx_data;
15    wire rx_valid;
16    wire rx_framing_error;
17    wire tx_ready;
18    reg tx_valid;
19    reg [7:0] tx_data;
20
21    reg [7:0] last_data;
22
23    uart_rx #(DIV/DIV) u_rx(
24        .clk(clk_50mhz), .rst_p(rst_p), .rx(uart_rxd), .data(rx_data),
25        .valid(rx_valid), .framing_error(rx_framing_error));
26    uart_tx #(DIV/DIV) u_tx(
27        .clk(clk_50mhz), .rst_p(rst_p), .valid(tx_valid), .data(tx_data),
28        .ready(tx_ready), .tx(uart_txd));
29
30    always @(posedge clk_50mhz or posedge rst_p) begin
31        if (rst_p) begin
32            tx_valid <= 0; tx_data <= 0; last_data <= 0;
33        end else begin
34            tx_valid <= 0;
35            if (rx_valid) begin
36                last_data <= rx_data;
37                if (tx_ready) begin
38                    tx_data <= rx_data;
39                    tx_valid <= 1;
40                end
41            end
42        end
43    end
44
45    assign led = last_data;
46 endmodule

```

TOP Module, RX와 TX를 연결하여 Echo 기능 구현

· TB

```

1 timescale 1ns/1ps
2 module tb_uart_echo;
3     localparam integer DIV=8;
4     reg clk_50mhz=0,rst_p=1,uart_rxd=1;
5     wire uart_txd;
6     wire [7:0] led;
7     integer checks=0,1;
8     reg [7:0] observed;
9     always #10 clk_50mhz<=clk_50mhz;
10    lab3_uart_echo #(CLK_HZ(800),BAUD(100)) dut
11        (.clk_50mhz(clk_50mhz),.rst_p(rst_p),uart_rxd(uart_rxd),uart_txd(uart_txd),led[led]);
12
13    task send_byte(input [7:0] value);
14    begin
15        uart_rxd<=0;repeat(DIV@(posedge clk_50mhz);
16        for(i=0;i<8;i=i+1)begin uart_rxd<=value[i];repeat(DIV@(posedge clk_50mhz);end
17        uart_rxd<=1;repeat(DIV@(posedge clk_50mhz);
18    end endtask
19
20    task receive_byte(input [7:0] expected);
21    begin
22        observed<=0@(negedge uart_txd);repeat(DIV+DIV/2@(posedge clk_50mhz);
23        for(i=0;i<8;i=i+1)begin observed[i]<=uart_txd;repeat(DIV@(posedge clk_50mhz);end
24        if(observed!~=expected)$fatal(1,"echo-0h expected-0h",observed,expected);
25        if(led!~=expected)$fatal(1,"led-0h expected-0h",led,expected);
26        checks=checks+1;repeat(DIV@(posedge clk_50mhz);
27    end endtask
28
29    initial begin
30        $dumpfile("wave.vcd");$dumpvars(0,tb_uart_echo);
31        repeat(4@(posedge clk_50mhz);rst_p<=0;
32        fork send_byte(8'h41); receive_byte(8'h41); join
33        fork send_byte(8'h5a); receive_byte(8'h5a); join
34        fork send_byte(8'h0a); receive_byte(8'h0a); join
35        if(dut.rx_framing_error)$fatal(1,"unexpected framing error");
36        $display("LAB3_UART_ECHO_PASS checks=%0u",checks);$finish;
37    end
38
39    initial begin #50000;$fatal(1,"timeout");end
40 endmodule

```

• XDC



순서	입력 데이터	ASCII	기대 UART 출력	기대 LED
1	0x41	A	0x41	0x41
2	0x5A	Z	0x5A	0x5A
3	0x0A	LF	0x0A	0x0A

검사 항목	기대 결과
Echo 데이터 일치	PASS
LED 표시값 일치	PASS
Framing Error 발생 여부	미발생
check 카운트	3
최종 출력	LAB3_UART_ECHO_PASS checks=3

[illegible][illegible]

상태	변경 내용	결과
정상	BAUD=9600	PASS
수정	TX 또는 RX의 BAUD 값을 다르게 설정	FAIL 또는 데이터 불일치
복구	BAUD 원래 값으로 복원	PASS

- Start bit가 충분한 시간 동안 유지되어야 한다.

- Stop bit는 반드시 High 상태여야 한다.
- RX와 TX의 Baud Rate가 동일해야 한다.
- TX가 Busy 상태일 때 새로운 데이터를 전송하면 데이터 손실이 발생할 수 있다.
- framing_error가 발생하지 않아야 한다.

- 보드에서 확인해야 할 항목과 예상 결과
- 확인 항목

확인 내용	예상 결과
50MHz 클럭 연결	정상 동작
Reset 버튼 동작	LED=0x00
UART RX 연결	데이터 수신 가능
UART TX 연결	예고 데이터 송신
LED 출력	마지막 수신 데이터 표시
9600 8N1 설정	정상 통신
Local Echo OFF	FPGA Echo만 표시

- 예상 결과

PC 터미널에서 문자를 입력하면 FPGA가 동일한 문자를 즉시 다시 송신해야 한다. 예를 들어 'A'를 입력하면 터미널에 'A'가 다시 출력되고 LED는 0x41 패턴을 표시해야 한다. 이후 'Z'를 입력하면 LED 값은 0x5A로 변경된다. 모든 문자에 대해 입력 데이터와 출력 데이터가 동일하며 framing error 없이 정상 동작하는 것이 실험의 기대 결과이다.

8. 참고 문헌

- 전자전기컴퓨터설계실험 II 교안
- Microchip UART 공식 자료 - Microchip Technology, Universal Asynchronous Receiver and Transmitter (UART) Peripherals.
- A. M. et al., Design and Implementation of UART With

Effective Serial Communication, IEEE, 2024. Available via IEEE Xplore.

9. Git-hub 주소

<https://github.com/UnnamedDays/ECE-II-Project>